Assignment No: - 10

Name :- Isha Ghorpade

Enrollnment :- 22420020

Roll :- 381066

Problem Statement

Implement an auto-complete system using the N-gram language model to predict the next word based on previous words.

Objectives:

To study and implement the N-gram model for text prediction.
To learn how probability is used to predict the next word.
To build an auto-complete system using N-grams.
To analyze how context improves prediction accuracy.

S/W Packages and H/W apparatus used

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Python 3.x
- **Development Tools:** Jupyter Notebook / Google Colab / VS Code
- **Library Used:** NLTK (Natural Language Toolkit)

Hardware Apparatus Used

- Computer or Laptop
- Processor: Intel / AMD or equivalent
- Minimum **8 GB RAM**
- Internet connection (for downloading NLTK resources)

THEORY :

Auto-complete System

An auto-complete system predicts the next word or phrase based on the previously typed text. It is widely used in:

- Search engines
- Messaging apps
- Code editors

It improves typing speed and user experience.

N-gram Model

An N-gram is a sequence of **N consecutive words** from a text.

Examples:

- Unigram (1-gram): *I, love, NLP*
- Bigram (2-gram): *I love, love NLP*
- Trigram (3-gram): *I love NLP*

Example

Input: *I love*
Possible predictions:

- I love **NLP**
- I love **coding**

The model selects the word with **highest frequency/probability**.

Advantages

• Simple and easy to implement
• Fast prediction
• Works well for small datasets

Limitations

• Cannot capture long context
• Data sparsity problem
• Accuracy depends on dataset size

Algorithm (Steps)

1. Input training text
2. Tokenize the text into words
3. Generate N-grams (bigram/trigram)
4. Count frequency of N-grams
5. Calculate probabilities
6. Given input words, predict next word with highest probability

---

Python Code (Auto-complete using N-gram)

```
import nltk
from nltk.util import ngrams
from collections import defaultdict, Counter

# Sample dataset
text = """I love NLP and I love machine learning.
NLP is fun and machine learning is powerful.
I love coding and I love AI."""

# Preprocessing
tokens = nltk.word_tokenize(text.lower())

# Create bigrams
n = 2
bigrams = list(ngrams(tokens, n))

# Build model
model = defaultdict(Counter)

for w1, w2 in bigrams:
    model[w1][w2] += 1

# Function for prediction
def predict_next_word(word):
    if word in model:
        return model[word].most_common(3)
    else:
        return "No prediction available"

# Test
```

```
input_word = "love"
print(f"Next words for '{input_word}':")
print(predict_next_word(input_word))
```

Output Example

```
Next words for 'love':
[('nlp', 2), ('machine', 1), ('coding', 1)]
```

Improved Version (Trigram for Better Accuracy)

```
# Create trigrams
n = 3
trigrams = list(ngrams(tokens, n))

model_tri = defaultdict(Counter)

for w1, w2, w3 in trigrams:
    model_tri[(w1, w2)][w3] += 1

def predict_next_word_trigram(w1, w2):
    if (w1, w2) in model_tri:
        return model_tri[(w1, w2)].most_common(3)
    else:
        return "No prediction available"

# Test
print(predict_next_word_trigram("i", "love"))
```

Conclusion

The N-gram model is a simple and effective method for building an auto-complete system. It predicts the next word based on previous words using probability. While it works well for small datasets, it has limitations in handling long context. However, it forms the foundation for advanced language models used in modern NLP applications.